

Building Technology That People Can Trust: Why AI Responsibility Matters

Building Technology That People Can Trust: Why AI Responsibility Matters

Everyone talks about what AI can do. Not enough people talk about what happens when it goes wrong.

As a computer science student, I have become increasingly interested in how artificial intelligence is shaping the future of technology. AI systems are now used in healthcare (such as diagnostic tools), finance (fraud detection), education, and many other parts of everyday life. While these tools can improve efficiency and support decision-making, they also raise important concerns about privacy, bias, misuse, and security. Because of this, I believe conversations about AI should not only focus on innovation, but also on responsibility.

One thing I keep noticing is that AI tends to get celebrated for what it can do before anyone really stops to ask what could go wrong. In tech, there is this push to build fast and fix problems later. But that mindset does not work well with AI. When a system is built without thinking about security from the beginning, it becomes vulnerable to data leaks, manipulation, and misuse. And the bigger the system, the harder it is to fix once it is already out there. By then, real people's information is at risk. The damage is already done. One example of how governments are trying to regulate AI is shown in the EU's risk-based framework below.

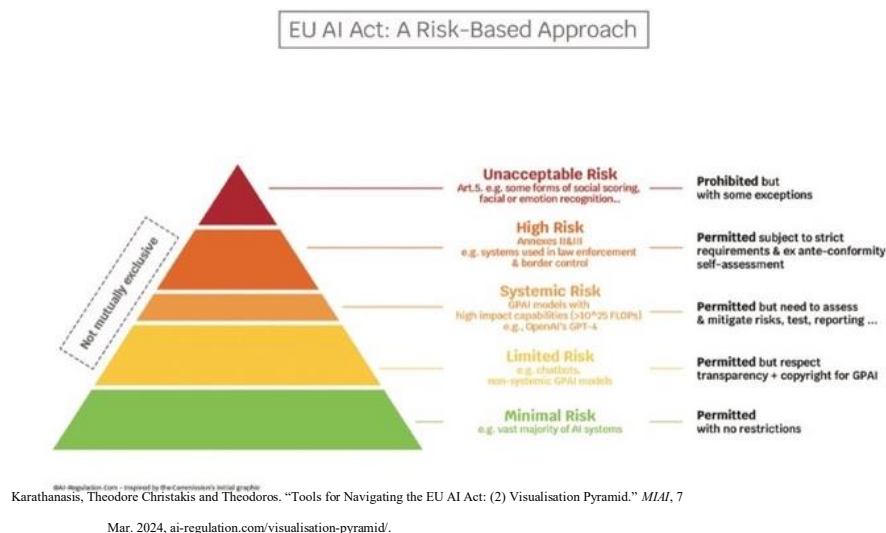


Figure 1: The EU AI Act's risk-based framework categorizes AI systems by level of risk, highlighting why security, oversight, and responsible design are critical in higher-risk applications.

This model classifies AI systems based on risk levels, which shows how seriously policymakers are starting to treat safety and responsibility

Honestly, before I started learning about cybersecurity, I thought of it as its own separate thing, something specialists handled, not something every developer needed to worry about. That changed quickly. Security is not a feature you add at the end. It must be part of how you think about a project from day one. Things like how data is collected, who has access to it, and how the system could be attacked are questions that should come up early, not after something goes

wrong. Waiting until the end to think about security is like building a house and then deciding whether to put locks on the doors.

There are frameworks that try to help with this. The [NIST Cybersecurity Framework](#), for example, gives organizations a structured way to think about protecting their systems and data. But a framework only helps if the people building the systems actually care about using it. That has to start with individual developers taking responsibility, not just following a checklist because they must.

The consequences of getting this wrong are not small. For instance, AI-driven systems that have been hacked on a large scale have leaked the personal information of millions of users, showing how just one security breach can have real-world effects. AI systems often handle sensitive information like medical records, financial data, personal details and they process it fast and at a huge scale. One security gap can cause damage that would not have been possible with older technology. This is not just a theoretical concern either. There have already been security breaches involving automated and AI-driven systems, and in some cases, they have put the data of millions of users at risk. The stakes are high, and I think that means developers need to treat security as a core responsibility, not an optional extra.

Studying this has changed how I think about what it means to be a good developer. It is not just about writing clean code or building something that works. It is about thinking through who uses it, what they are trusting you with, and what could go wrong if you are not careful. That kind of thinking takes more than technical skill, it takes accountability. I think that is something the field as a whole is still figuring out how to teach. It is not really covered in a lot of intro courses, which I think is a gap worth paying attention to.

For students getting into this field, I think that means building a security-first mindset alongside everything else. Knowing how to build something is important but so is understanding what you are responsible for once it is out in the world. As AI keeps growing and getting embedded into more critical systems, I think the developers who take that seriously will be the ones who actually make a difference.

At the end of the day, the most advanced system is not always the best one. The best one is the one people can trust. And that does not happen by accident, it has to be built in from the start.

What do you think — should the tech industry be doing more to prioritize security before releasing AI systems? I would like to hear other perspectives on this.